

BASE DE DATOS II

Práctica de Laboratorio y Análisis

Bases de Datos Distribuidas

Transacciones y Control de Concurrencia

Transacciones distribuidas | Bloqueos | Aislamiento | Conflictos | PostgreSQL

Campo	Descripción
Curso	Base de Datos II
Nivel	Universitario
Duración sugerida	2 a 3 horas
Modalidad	Individual o grupal
Herramienta	PostgreSQL
Entregable	Informe técnico con evidencias de ejecución y análisis
Sesión	Transacciones y Control de Concurrencia

Índice de la práctica

1. Presentación de la práctica
2. Logro de aprendizaje
3. Competencias a desarrollar
4. Caso de estudio
5. Parte I: Preparación de la base de datos
6. Parte II: Inserción de datos de prueba
7. Parte III: Simulación de una transacción distribuida
8. Parte IV: Uso de ROLLBACK ante error
9. Parte V: Control de concurrencia con dos sesiones
10. Parte VI: Comparación de niveles de aislamiento
11. Parte VII: Simulación de conflicto por sobreventa
12. Parte VIII: Localización de nodos
13. Parte IX: Auditoría de transacciones
14. Parte X: Diseño propio de control de concurrencia
15. Entregable final
16. Rúbrica de evaluación
17. Anexo A: Script SQL completo

1. Presentación de la práctica

La presente práctica tiene como finalidad que el estudiante simule transacciones en una base de datos distribuida utilizando PostgreSQL. El laboratorio permite analizar cómo una transacción puede modificar múltiples registros relacionados, como descontar inventario, registrar una compra, actualizar el estado de un pedido y registrar un pago.

Además, se implementará control de concurrencia mediante sesiones simultáneas, con el objetivo de observar cómo PostgreSQL gestiona conflictos cuando varias transacciones intentan acceder o modificar los mismos datos al mismo tiempo.

Importante: Aunque PostgreSQL se ejecutará en un entorno local, la práctica simula una base de datos distribuida mediante esquemas que representan nodos lógicos.

2. Logro de aprendizaje

Al finalizar la práctica, el estudiante implementa transacciones en un entorno distribuido simulado, aplica mecanismos de control de concurrencia y analiza conflictos generados por accesos simultáneos a los mismos datos.

3. Competencias a desarrollar

- Comprender el concepto de transacción en bases de datos distribuidas.
- Aplicar propiedades ACID en operaciones de compra e inventario.
- Simular nodos distribuidos mediante esquemas en PostgreSQL.
- Ejecutar transacciones que modifiquen múltiples tablas.
- Implementar control de concurrencia usando bloqueos.
- Comparar comportamientos con distintos niveles de aislamiento.
- Analizar conflictos entre transacciones concurrentes.
- Interpretar resultados mediante consultas de verificación.

4. Caso de estudio

La empresa Comercial Andina S.A.C. administra una plataforma de ventas en línea. Su sistema se encuentra distribuido en diferentes nodos lógicos:

Nodo lógico	Responsabilidad
nodo_inventario	Control de productos y stock
nodo_pedidos	Registro y seguimiento de pedidos
nodo_pagos	Registro de pagos
nodo_logistica	Control de envíos

Cuando un cliente realiza una compra, el sistema debe verificar stock, descontar inventario, crear el pedido, registrar el detalle, procesar el pago y actualizar el estado del pedido. El problema principal aparece cuando dos o más clientes intentan comprar el mismo producto al mismo tiempo.

5. Parte I: Preparación de la base de datos

5.1 Creación de la base de datos

```
CREATE DATABASE bd_transacciones_distribuidas;
```

```
-- Conectarse desde psql:
\c bd_transacciones_distribuidas;
```

5.2 Creación de esquemas para simular nodos distribuidos

```
CREATE SCHEMA nodo_inventario;
CREATE SCHEMA nodo_pedidos;
CREATE SCHEMA nodo_pagos;
CREATE SCHEMA nodo_logistica;
```

5.3 Creación de tablas base

```
CREATE TABLE nodo_inventario.productos (
    id_producto SERIAL PRIMARY KEY,
    nombre_producto VARCHAR(100) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock INT NOT NULL CHECK (stock >= 0),
    nodo_asignado VARCHAR(50) NOT NULL
);

CREATE TABLE nodo_pedidos.clientes (
    id_cliente SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    dni CHAR(8) UNIQUE NOT NULL,
    correo VARCHAR(100),
    ciudad VARCHAR(50)
);

CREATE TABLE nodo_pedidos.pedidos (
    id_pedido SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(30) NOT NULL,
    total NUMERIC(10,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (id_cliente) REFERENCES nodo_pedidos.clientes(id_cliente)
);

CREATE TABLE nodo_pedidos.detalle_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    precio_unitario NUMERIC(10,2) NOT NULL,
    subtotal NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (id_pedido) REFERENCES nodo_pedidos.pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES nodo_inventario.productos(id_producto)
);

CREATE TABLE nodo_pagos.pagos (
    id_pago SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    metodo_pago VARCHAR(50) NOT NULL,
    monto NUMERIC(10,2) NOT NULL,
    estado_pago VARCHAR(30) NOT NULL,
    fecha_pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_pedido) REFERENCES nodo_pedidos.pedidos(id_pedido)
);

CREATE TABLE nodo_logistica.envios (
    id_envio SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    ciudad_destino VARCHAR(50) NOT NULL,
    estado_envio VARCHAR(30) NOT NULL,
    fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (id_pedido) REFERENCES nodo_pedidos.pedidos(id_pedido)
);
```

6. Parte II: Inserción de datos de prueba

```
INSERT INTO nodo_inventario.productos
(nombre_producto, categoria, precio, stock, nodo_asignado)
VALUES
('Laptop Lenovo IdeaPad', 'Tecnología', 2800.00, 5, 'Nodo_Inventario_Lima'),
('Mouse Logitech', 'Accesorios', 85.00, 20, 'Nodo_Inventario_Lima'),
('Teclado Mecánico Redragon', 'Accesorios', 180.00, 10, 'Nodo_Inventario_Arequipa'),
('Monitor Samsung 24', 'Tecnología', 650.00, 3, 'Nodo_Inventario_Trujillo');

INSERT INTO nodo_pedidos.clientes
(nombres, apellidos, dni, correo, ciudad)
VALUES
('Luis', 'Ramírez', '12345678', 'luis.ramirez@mail.com', 'Trujillo'),
('María', 'Torres', '23456789', 'maria.torres@mail.com', 'Arequipa'),
('Carlos', 'Mendoza', '34567890', 'carlos.mendoza@mail.com', 'Lima'),
('Ana', 'Paredes', '45678901', 'ana.paredes@mail.com', 'Iquitos');
```

7. Parte III: Simulación de una transacción distribuida

7.1 Objetivo

Crear una transacción que simule una compra y modifique registros ubicados en diferentes nodos lógicos: inventario, pedidos, pagos y logística.

7.2 Transacción de compra exitosa

```
BEGIN;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 1
FOR UPDATE;

UPDATE nodo_inventario.productos
SET stock = stock - 2
WHERE id_producto = 1
AND stock >= 2;

INSERT INTO nodo_pedidos.pedidos (id_cliente, estado, total)
VALUES (1, 'Registrado', 5600.00)
RETURNING id_pedido;

-- Supongamos que el id_pedido generado fue 1
INSERT INTO nodo_pedidos.detalle_pedido
(id_pedido, id_producto, cantidad, precio_unitario, subtotal)
VALUES (1, 1, 2, 2800.00, 5600.00);

INSERT INTO nodo_pagos.pagos
(id_pedido, metodo_pago, monto, estado_pago)
VALUES (1, 'Tarjeta', 5600.00, 'Aprobado');

INSERT INTO nodo_logistica.envios
(id_pedido, ciudad_destino, estado_envio)
VALUES (1, 'Trujillo', 'Pendiente');

UPDATE nodo_pedidos.pedidos
```

```
SET estado = 'Pagado'
WHERE id_pedido = 1;

COMMIT;
```

7.3 Verificación de resultados

```
SELECT * FROM nodo_inventario.productos;
SELECT * FROM nodo_pedidos.pedidos;
SELECT * FROM nodo_pedidos.detalle_pedido;
SELECT * FROM nodo_pagos.pagos;
SELECT * FROM nodo_logistica.envios;
```

7.4 Actividades de análisis

1. ¿Qué tablas fueron modificadas durante la transacción?
2. ¿Qué nodos lógicos participaron en la operación?
3. ¿Por qué se utilizó BEGIN y COMMIT?
4. ¿Qué ocurriría si falla el registro del pago después de descontar el inventario?
5. ¿Por qué es importante que todas las operaciones se ejecuten como una sola unidad lógica?

8. Parte IV: Uso de ROLLBACK ante error

8.1 Simulación de una compra fallida

```
BEGIN;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4
FOR UPDATE;

UPDATE nodo_inventario.productos
SET stock = stock - 10
WHERE id_producto = 4
AND stock >= 10;

-- Si no se actualizó ninguna fila, cancelar la transacción
ROLLBACK;
```

8.2 Verificación posterior

```
SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4;
```

8.3 Actividades de análisis

6. ¿Por qué la transacción debe cancelarse si no existe stock suficiente?
7. ¿Qué función cumple ROLLBACK?
8. ¿Qué inconsistencia aparecería si se registrara el pedido sin descontar inventario?
9. ¿Cómo se relaciona esta operación con la atomicidad?
10. ¿Qué validación adicional implementaría para evitar errores de stock?

9. Parte V: Control de concurrencia con dos sesiones

Para esta parte se deben abrir dos ventanas de psql o dos pestañas de consulta en pgAdmin: Sesión A y Sesión B.

9.1 Consulta inicial del producto

```
SELECT id_producto, nombre_producto, stock
FROM nodo_inventario.productos
WHERE id_producto = 4;
```

9.2 Sesión A: bloquear el producto

```
BEGIN;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4
FOR UPDATE;

-- No ejecutar COMMIT todavía.
```

9.3 Sesión B: intentar modificar el mismo producto

```
BEGIN;

UPDATE nodo_inventario.productos
SET stock = stock - 1
WHERE id_producto = 4;

-- Esta operación quedará esperando.
```

9.4 Sesión A: liberar el bloqueo

```
COMMIT;
```

9.5 Sesión B: confirmar operación

```
COMMIT;
```

9.6 Verificación final

```
SELECT id_producto, nombre_producto, stock
FROM nodo_inventario.productos
WHERE id_producto = 4;
```

9.7 Actividades de análisis

11. ¿Qué ocurrió cuando la Sesión B intentó modificar el registro bloqueado?
12. ¿Por qué la Sesión B tuvo que esperar?
13. ¿Qué tipo de bloqueo generó SELECT ... FOR UPDATE?
14. ¿Qué problema se evita al bloquear el registro antes de actualizarlo?
15. ¿Qué pasaría si ambas sesiones modificaran el stock sin control de concurrencia?

10. Parte VI: Comparación de niveles de aislamiento

10.1 Nivel READ COMMITTED

```
-- Sesión A
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT stock
FROM nodo_inventario.productos
```

```

WHERE id_producto = 2;

-- Sesión B
BEGIN;
UPDATE nodo_inventario.productos
SET stock = stock - 5
WHERE id_producto = 2;
COMMIT;

-- Regresar a Sesión A
SELECT stock
FROM nodo_inventario.productos
WHERE id_producto = 2;
COMMIT;

```

10.2 Nivel REPEATABLE READ

```

-- Sesión A
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT stock
FROM nodo_inventario.productos
WHERE id_producto = 2;

-- Sesión B
BEGIN;
UPDATE nodo_inventario.productos
SET stock = stock - 2
WHERE id_producto = 2;
COMMIT;

-- Regresar a Sesión A
SELECT stock
FROM nodo_inventario.productos
WHERE id_producto = 2;
COMMIT;

```

10.3 Actividades de análisis

16. ¿Qué diferencia se observa entre READ COMMITTED y REPEATABLE READ?
17. ¿En qué nivel de aislamiento la Sesión A puede observar cambios confirmados por otra transacción?
18. ¿En qué nivel de aislamiento la Sesión A mantiene una vista estable de los datos?
19. ¿Qué nivel de aislamiento sería más adecuado para una operación crítica de inventario?
20. ¿Qué desventaja puede tener usar niveles de aislamiento más estrictos?

11. Parte VII: Simulación de conflicto por sobreventa

11.1 Preparación del producto

```

UPDATE nodo_inventario.productos
SET stock = 1
WHERE id_producto = 4;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4;

```


11.2 Sesión A: cliente intenta comprar el último producto

```
BEGIN;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4
FOR UPDATE;

UPDATE nodo_inventario.productos
SET stock = stock - 1
WHERE id_producto = 4
AND stock >= 1;

-- No confirmar todavía.
```

11.3 Sesión B: otro cliente intenta comprar el mismo producto

```
BEGIN;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4
FOR UPDATE;

-- Esta consulta esperará hasta que la Sesión A confirme o cancele.
```

11.4 Confirmación y verificación

```
-- Sesión A
COMMIT;

-- Sesión B
UPDATE nodo_inventario.productos
SET stock = stock - 1
WHERE id_producto = 4
AND stock >= 1;
COMMIT;

SELECT *
FROM nodo_inventario.productos
WHERE id_producto = 4;
```

11.5 Actividades de análisis

21. ¿La Sesión B logró descontar inventario?
22. ¿Por qué la condición AND stock >= 1 es importante?
23. ¿Qué problema de concurrencia se evitó?
24. ¿Qué ocurriría si se eliminara la condición de validación de stock?
25. ¿Cómo se relaciona este caso con la consistencia de una base de datos distribuida?

12. Parte VIII: Tabla de localización de nodos

12.1 Creación de tabla de localización

```
CREATE TABLE localizacion_nodos (
    id_localizacion SERIAL PRIMARY KEY,
    esquema VARCHAR(100) NOT NULL,
    tabla VARCHAR(100) NOT NULL,
    nodo_logico VARCHAR(100) NOT NULL,
```

```
ubicacion_geografica VARCHAR(100) NOT NULL,
funcion_principal VARCHAR(150) NOT NULL
);
```

12.2 Inserción y consulta

```
INSERT INTO localizacion_nodos
(esquema, tabla, nodo_logico, ubicacion_geografica, funcion_principal)
VALUES
('nodo_inventario', 'productos', 'Nodo_Inventario', 'Lima', 'Control de stock y productos'),
('nodo_pedidos', 'clientes', 'Nodo_Pedidos', 'Trujillo', 'Gestión de clientes'),
('nodo_pedidos', 'pedidos', 'Nodo_Pedidos', 'Trujillo', 'Registro de pedidos'),
('nodo_pedidos', 'detalle_pedido', 'Nodo_Pedidos', 'Trujillo', 'Detalle de productos vendidos'),
('nodo_pagos', 'pagos', 'Nodo_Pagos', 'Lima', 'Procesamiento de pagos'),
('nodo_logistica', 'envios', 'Nodo_Logistica', 'Arequipa', 'Gestión de envíos');

SELECT * FROM localizacion_nodos;
```

12.3 Actividades de análisis

26. ¿Por qué es importante conocer la ubicación lógica de cada tabla?
27. ¿Qué nodos participaron en la transacción de compra?
28. ¿Qué nodo debería tener mayor control de concurrencia?
29. ¿Qué riesgos aparecen si el nodo de inventario falla?
30. ¿Cómo afectaría la latencia entre nodos a una transacción distribuida?

13. Parte IX: Consulta de auditoría de transacciones

```
CREATE TABLE auditoria_transacciones (
  id_auditoria SERIAL PRIMARY KEY,
  operacion VARCHAR(100) NOT NULL,
  tabla_afectada VARCHAR(100) NOT NULL,
  descripcion TEXT NOT NULL,
  fecha_operacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO auditoria_transacciones
(operacion, tabla_afectada, descripcion)
VALUES
('COMPRA', 'productos, pedidos, detalle_pedido, pagos, envios',
'Se registró una compra distribuida con descuento de inventario, pedido, pago y envío.');
```

```
SELECT *
FROM auditoria_transacciones
ORDER BY fecha_operacion DESC;
```

13.1 Actividades de análisis

31. ¿Qué utilidad tiene una tabla de auditoría en una base de datos distribuida?
32. ¿Qué información mínima debería registrar una auditoría de transacciones?
33. ¿Cómo ayudaría la auditoría a detectar errores de concurrencia?
34. ¿Qué diferencia existe entre registrar auditoría manualmente y mediante triggers?
35. ¿Por qué la auditoría es importante en operaciones críticas como pagos e inventario?

14. Parte X: Diseño propio de control de concurrencia

La empresa espera manejar miles de compras simultáneas durante campañas de descuento. Los productos más vendidos pueden ser comprados por muchos clientes al mismo tiempo. Complete la siguiente tabla:

Escenario	Problema de concurrencia posible	Mecanismo propuesto	Justificación
Compra simultánea del último producto			
Pago duplicado de un pedido			
Actualización del estado de envío			
Cancelación de pedido ya pagado			
Modificación simultánea de precio			

14.1 Preguntas de análisis avanzado

36. ¿Qué propiedades ACID se evidencian en la transacción de compra?
37. ¿Por qué una transacción distribuida es más compleja que una transacción local?
38. ¿Qué problema resuelve el bloqueo FOR UPDATE?
39. ¿Qué es una actualización perdida?
40. ¿Qué es una lectura no repetible?
41. ¿Qué es una lectura fantasma?
42. ¿Por qué puede aparecer bloqueo entre transacciones?
43. ¿Qué diferencia existe entre bloqueo pesimista y bloqueo optimista?
44. ¿Qué impacto tiene el control de concurrencia en el rendimiento?
45. ¿Qué estrategia recomendaría para evitar sobreventa en un sistema real?
46. ¿Qué aprendió al simular transacciones concurrentes en PostgreSQL?

15. Entregable final

- Carátula: nombre del estudiante, curso, docente, fecha y título de la práctica.
- Introducción: bases de datos distribuidas, transacciones y control de concurrencia.
- Desarrollo en PostgreSQL: creación de esquemas, tablas, datos, transacciones y pruebas concurrentes.
- Evidencias: capturas de ejecución de cada parte.
- Análisis: respuestas a las preguntas planteadas.
- Comparación: comportamiento de diferentes niveles de aislamiento.
- Propuesta técnica: estrategia de control de concurrencia para compras simultáneas.
- Conclusiones: mínimo tres conclusiones relacionadas con transacciones, concurrencia y consistencia.
- Anexos: script SQL completo y capturas adicionales.

16. Rúbrica de evaluación

Criterio	Excelente (4 pts)	Bueno (3 pts)	Regular (2 pts)	Deficiente (1 pt)
Comprensión conceptual	Explica claramente transacciones, ACID, concurrencia y aislamiento.	Explica la mayoría de conceptos correctamente.	Presenta explicaciones incompletas.	Presenta errores conceptuales graves.
Implementación en PostgreSQL	Crea correctamente esquemas, tablas, transacciones y pruebas concurrentes.	Presenta pequeños errores corregibles.	Implementación parcial.	No logra implementar adecuadamente.

Control de concurrencia	Demuestra bloqueos, espera entre sesiones y validación de stock.	Demuestra la mayoría de pruebas.	Evidencia limitada de concurrencia.	No evidencia control de concurrencia.
Análisis de resultados	Interpreta correctamente conflictos, bloqueos y niveles de aislamiento.	Analiza resultados de forma general.	Análisis superficial.	No analiza resultados.
Informe final	Presentación profesional, ordenada y completa.	Presentación clara con mínimos errores.	Presentación desordenada o incompleta.	No cumple estructura solicitada.

17. Anexo A: Script SQL completo

El siguiente script integra la preparación de la base de datos, creación de esquemas, tablas, datos de prueba, transacción de compra, ROLLBACK, localización de nodos y auditoría.

```
-- =====
-- PRACTICA: TRANSACCIONES Y CONTROL DE CONCURRENCIA
-- CURSO: BASE DE DATOS II
-- MOTOR: POSTGRESQL
-- =====

CREATE DATABASE bd_transacciones_distribuidas;
-- \c bd_transacciones_distribuidas;

CREATE SCHEMA nodo_inventario;
CREATE SCHEMA nodo_pedidos;
CREATE SCHEMA nodo_pagos;
CREATE SCHEMA nodo_logistica;

CREATE TABLE nodo_inventario.productos (
    id_producto SERIAL PRIMARY KEY,
    nombre_producto VARCHAR(100) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock INT NOT NULL CHECK (stock >= 0),
    nodo_asignado VARCHAR(50) NOT NULL
);

CREATE TABLE nodo_pedidos.clientes (
    id_cliente SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    dni CHAR(8) UNIQUE NOT NULL,
    correo VARCHAR(100),
    ciudad VARCHAR(50)
);

CREATE TABLE nodo_pedidos.pedidos (
    id_pedido SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(30) NOT NULL,
    total NUMERIC(10,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (id_cliente) REFERENCES nodo_pedidos.clientes(id_cliente)
);

CREATE TABLE nodo_pedidos.detalle_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    precio_unitario NUMERIC(10,2) NOT NULL,
```

```

    subtotal NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (id_pedido) REFERENCES nodo_pedidos.pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES nodo_inventario.productos(id_producto)
);

CREATE TABLE nodo_pagos.pagos (
    id_pago SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    metodo_pago VARCHAR(50) NOT NULL,
    monto NUMERIC(10,2) NOT NULL,
    estado_pago VARCHAR(30) NOT NULL,
    fecha_pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_pedido) REFERENCES nodo_pedidos.pedidos(id_pedido)
);

CREATE TABLE nodo_logistica.envios (
    id_envio SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    ciudad_destino VARCHAR(50) NOT NULL,
    estado_envio VARCHAR(30) NOT NULL,
    fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_pedido) REFERENCES nodo_pedidos.pedidos(id_pedido)
);

INSERT INTO nodo_inventario.productos
(nombre_producto, categoria, precio, stock, nodo_asignado)
VALUES
('Laptop Lenovo IdeaPad', 'Tecnología', 2800.00, 5, 'Nodo_Inventario_Lima'),
('Mouse Logitech', 'Accesorios', 85.00, 20, 'Nodo_Inventario_Lima'),
('Teclado Mecánico Redragon', 'Accesorios', 180.00, 10, 'Nodo_Inventario_Arequipa'),
('Monitor Samsung 24"', 'Tecnología', 650.00, 3, 'Nodo_Inventario_Trujillo');

INSERT INTO nodo_pedidos.clientes
(nombres, apellidos, dni, correo, ciudad)
VALUES
('Luis', 'Ramírez', '12345678', 'luis.ramirez@mail.com', 'Trujillo'),
('María', 'Torres', '23456789', 'maria.torres@mail.com', 'Arequipa'),
('Carlos', 'Mendoza', '34567890', 'carlos.mendoza@mail.com', 'Lima'),
('Ana', 'Paredes', '45678901', 'ana.paredes@mail.com', 'Iquitos');

BEGIN;
SELECT * FROM nodo_inventario.productos
WHERE id_producto = 1
FOR UPDATE;
UPDATE nodo_inventario.productos
SET stock = stock - 2
WHERE id_producto = 1 AND stock >= 2;
INSERT INTO nodo_pedidos.pedidos (id_cliente, estado, total)
VALUES (1, 'Registrado', 5600.00);
INSERT INTO nodo_pedidos.detalle_pedido
(id_pedido, id_producto, cantidad, precio_unitario, subtotal)
VALUES (1, 1, 2, 2800.00, 5600.00);
INSERT INTO nodo_pagos.pagos
(id_pedido, metodo_pago, monto, estado_pago)
VALUES (1, 'Tarjeta', 5600.00, 'Aprobado');
INSERT INTO nodo_logistica.envios
(id_pedido, ciudad_destino, estado_envio)
VALUES (1, 'Trujillo', 'Pendiente');
UPDATE nodo_pedidos.pedidos
SET estado = 'Pagado'
WHERE id_pedido = 1;
COMMIT;

SELECT * FROM nodo_inventario.productos;
SELECT * FROM nodo_pedidos.pedidos;
SELECT * FROM nodo_pedidos.detalle_pedido;
SELECT * FROM nodo_pagos.pagos;
SELECT * FROM nodo_logistica.envios;

```

```

BEGIN;
SELECT * FROM nodo_inventario.productos
WHERE id_producto = 4
FOR UPDATE;
UPDATE nodo_inventario.productos
SET stock = stock - 10
WHERE id_producto = 4 AND stock >= 10;
ROLLBACK;

CREATE TABLE localizacion_nodos (
    id_localizacion SERIAL PRIMARY KEY,
    esquema VARCHAR(100) NOT NULL,
    tabla VARCHAR(100) NOT NULL,
    nodo_logico VARCHAR(100) NOT NULL,
    ubicacion_geografica VARCHAR(100) NOT NULL,
    funcion_principal VARCHAR(150) NOT NULL
);

INSERT INTO localizacion_nodos
(esquema, tabla, nodo_logico, ubicacion_geografica, funcion_principal)
VALUES
('nodo_inventario', 'productos', 'Nodo_Inventario', 'Lima', 'Control de stock y productos'),
('nodo_pedidos', 'clientes', 'Nodo_Pedidos', 'Trujillo', 'Gestión de clientes'),
('nodo_pedidos', 'pedidos', 'Nodo_Pedidos', 'Trujillo', 'Registro de pedidos'),
('nodo_pedidos', 'detalle_pedido', 'Nodo_Pedidos', 'Trujillo', 'Detalle de productos vendidos'),
('nodo_pagos', 'pagos', 'Nodo_Pagos', 'Lima', 'Procesamiento de pagos'),
('nodo_logistica', 'envios', 'Nodo_Logistica', 'Arequipa', 'Gestión de envíos');

CREATE TABLE auditoria_transacciones (
    id_auditoria SERIAL PRIMARY KEY,
    operacion VARCHAR(100) NOT NULL,
    tabla_afectada VARCHAR(100) NOT NULL,
    descripcion TEXT NOT NULL,
    fecha_operacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO auditoria_transacciones
(operacion, tabla_afectada, descripcion)
VALUES
('COMPRA', 'productos, pedidos, detalle_pedido, pagos, envios',
'Compra distribuida con descuento de inventario, pedido, pago y envío.');
```

```

SELECT * FROM auditoria_transacciones ORDER BY fecha_operacion DESC;
```